Python time module

```
# ============================================================
# Python time Module
# ============================================================
#
# The time module is a built-in Python module used for:
#
# - Getting the current time
# - Unix timestamps
# - Pausing execution
# - Measuring execution time
# - Converting timestamps
# - Working with low-level time-related operations
#
# Import:
# ============================================================

import time


# ============================================================
# 1. time.time()
# ============================================================
#
# Returns the current time as a Unix timestamp.
#
# Unix timestamp = number of seconds since:
# January 1, 1970 00:00:00 UTC
#
# The result is a float because it can contain fractions
# of a second.

current_timestamp = time.time()

print("Current Unix timestamp:", current_timestamp)


# ============================================================
# 2. time.sleep()
# ============================================================
#
# Pauses the program for a specified number of seconds.
#
# This is very useful when:
#
# - Simulating streaming data
# - Waiting between API requests
# - Creating files periodically
# - Testing pipelines
# - Controlling loops

print("Before sleep")

time.sleep(2)

print("After 2 seconds")


# ============================================================
# 3. Measuring execution time
# ============================================================
#
# time.time() can be used to measure how long something takes.
#
# Start timer
# Execute operation
```

```python
# End timer
# Calculate elapsed time

start = time.time()

# Simulate some work
time.sleep(2)

end = time.time()

elapsed = end - start

print("Execution time:", elapsed, "seconds")


# ============================================================
# 4. time.perf_counter()
# ============================================================
#
# perf_counter() is designed for measuring elapsed time.
#
# It provides a high-resolution timer and is generally better
# than time.time() when benchmarking code.

start = time.perf_counter()

# Code we want to measure
total = 0

for i in range(1_000_000):
    total += i

end = time.perf_counter()

print("Loop execution time:", end - start, "seconds")


# ============================================================
# 5. time.monotonic()
# ============================================================
#
# Returns a clock that can only move forward.
#
# Useful for measuring durations because the clock is not
# affected by changes to the system clock.
#
# Example:
#
# start = time.monotonic()
# ...
# end = time.monotonic()
# elapsed = end - start
#
# Useful for:
# - Timeouts
# - Retry mechanisms
# - Long-running processes

start = time.monotonic()

time.sleep(1)

end = time.monotonic()

print("Duration:", end - start, "seconds")
```

```python
# ============================================================
# 6. time.localtime()
# ============================================================
#
# Converts a Unix timestamp into local time.
#
# If no timestamp is provided, it uses the current time.

local_time = time.localtime()

print("Local time:", local_time)


# ============================================================
# 7. Accessing parts of localtime()
# ============================================================
#
# localtime() returns a struct_time object.
#
# You can access:
#
# tm_year
# tm_mon
# tm_mday
# tm_hour
# tm_min
# tm_sec
# tm_wday
# tm_yday
# tm_isdst

now = time.localtime()

print("Year:", now.tm_year)
print("Month:", now.tm_mon)
print("Day:", now.tm_mday)
print("Hour:", now.tm_hour)
print("Minute:", now.tm_min)
print("Second:", now.tm_sec)


# ============================================================
# 8. time.gmtime()
# ============================================================
#
# Converts a Unix timestamp into UTC time.
#
# Difference:
#
# localtime() -> local timezone
# gmtime()    -> UTC

utc_time = time.gmtime()

print("UTC time:", utc_time)


# ============================================================
# 9. time.ctime()
# ============================================================
#
# Converts a Unix timestamp into a readable string.
#
# If no argument is provided, it uses the current time.

print("Readable time:", time.ctime())
```

```python
# ============================================================
# 10. time.strftime()
# ============================================================
#
# Formats a struct_time into a readable string.
#
# Common formatting codes:
#
# %Y -> Year
# %m -> Month
# %d -> Day
# %H -> Hour
# %M -> Minute
# %S -> Second

now = time.localtime()

formatted_time = time.strftime(
    "%Y-%m-%d %H:%M:%S",
    now
)

print("Formatted time:", formatted_time)


# ============================================================
# 11. time.strptime()
# ============================================================
#
# Does the opposite of strftime().
#
# strftime:
# struct_time -> string
#
# strptime:
# string -> struct_time

date_string = "2026-09-07 10:30:00"

parsed_time = time.strptime(
    date_string,
    "%Y-%m-%d %H:%M:%S"
)

print("Parsed time:", parsed_time)


# ============================================================
# 12. Convert struct_time to Unix timestamp
# ============================================================
#
# time.mktime() converts local struct_time into a Unix timestamp.

local_time = time.localtime()

timestamp = time.mktime(local_time)

print("Timestamp:", timestamp)


# ============================================================
# 13. Practical example:
#     Create data periodically
# ============================================================
#
# This pattern is useful when learning streaming systems.
#
# Imagine that a system receives data every 5 seconds.
#
```

```python
# You could use:
#
# while True:
#     create_data()
#     time.sleep(5)
#
# For learning, we will only run it 3 times.

for i in range(3):

    print("Generating data...")

    # Simulate data processing
    time.sleep(2)

    print("Data generated")


# ============================================================
# 14. Practical example:
#     Retry operation
# ============================================================
#
# time.sleep() is commonly used between retries.

for attempt in range(3):

    print("Attempt:", attempt + 1)

    # Simulate failure
    success = False

    if success:
        print("Success!")
        break

    print("Failed. Waiting before retry...")

    time.sleep(2)


# ============================================================
# 15. time module vs datetime module
# ============================================================
#
# time:
#
# - Unix timestamps
# - Sleeping
# - Measuring execution time
# - Low-level time operations
# - System/UTC time conversion
#
#
# datetime:
#
# - Dates
# - Times
# - Date + time
# - Date arithmetic
# - timedelta
# - Timezones
# - Parsing/formatting
#
#
# Example:
#
# time.time()
# -> timestamp
```

```
#
# datetime.now()
# -> datetime object
#
#
# In Data Engineering:
#
# time.sleep()
# -> wait between generated files
#
# time.time()
# -> timestamp
#
# time.perf_counter()
# -> measure pipeline performance
#
# time.strftime()
# -> format timestamps
#
# datetime
# -> more advanced date/time processing
```